PROYECTO II

Sistema de Administración de Estacionamiento Inteligente

Enunciado y especificación del proyecto

Curso: Programación II

Lenguaje: C++

Modalidad: Individual o en parejas (máximo dos integrantes)

Plazo de desarrollo: Dos semanas

Aplicación: Consola

Estructuras enlazadas · Iteradores · Memoria dinámica · Excepciones · Pruebas

1. Descripción general

Una empresa que administra estacionamientos necesita desarrollar un sistema para controlar el ingreso y salida de vehículos, organizar las solicitudes de estacionamiento, administrar los espacios disponibles y distribuir equitativamente las tareas entre sus encargados.

Actualmente, estos procesos se realizan manualmente, lo que ocasiona problemas como asignación incorrecta de espacios, pérdida de información y dificultades para identificar los vehículos estacionados.

Se solicita desarrollar una aplicación en C++ que permita administrar estas operaciones mediante distintas estructuras de datos dinámicas.

El sistema deberá incorporar:

- Una lista enlazada simple para administrar los vehículos que esperan ingresar.
- Una lista doblemente enlazada para controlar los vehículos estacionados.
- Una lista enlazada circular para administrar la rotación de los encargados del estacionamiento.
- Una matriz dinámica para representar la distribución física de los espacios.
- Iteradores propios para recorrer las estructuras enlazadas.
- Excepciones para detectar y gestionar operaciones inválidas.

Las estructuras deberán interactuar entre sí. Por ejemplo, cuando un vehículo pase de la lista de espera al estacionamiento, se deberá buscar un espacio disponible en la matriz, asignar el encargado correspondiente mediante la lista circular y registrar el vehículo en la lista doblemente enlazada.

2. Objetivo general

Desarrollar un sistema de administración de estacionamiento utilizando programación orientada a objetos, estructuras de datos enlazadas, matrices dinámicas, iteradores y manejo de excepciones en C++, que permita resolver diferentes operaciones de gestión y control de vehículos.

3. Requerimientos funcionales

3.1. Gestión de solicitudes de ingreso — Lista enlazada simple

Cuando un vehículo llega al estacionamiento, inicialmente debe registrarse en una lista de espera.

Cada solicitud deberá almacenar al menos:

- Número de placa.
- Nombre del propietario.
- Tipo de vehículo (automóvil o motocicleta).
- Número de solicitud, generado automáticamente.

Operaciones obligatorias:

1. Registrar una nueva solicitud al final de la lista.
2. Mostrar todas las solicitudes pendientes.
3. Consultar una solicitud mediante su número de placa.

4. Cancelar una solicitud, sin importar su posición.
5. Procesar la primera solicitud de la lista.
6. Consultar cuántas solicitudes permanecen pendientes.

Las solicitudes deberán atenderse en el mismo orden en que fueron registradas.

Consideraciones:

- No se pueden registrar placas duplicadas entre los vehículos en espera y los estacionados.
- Si se cancela la primera solicitud, la lista debe actualizar correctamente su inicio.
- La estructura debe funcionar cuando tenga cero, uno o múltiples elementos.
- No se permite perder solicitudes si el estacionamiento se encuentra lleno.

3.2. Control de vehículos estacionados — Lista doblemente enlazada

Una vez que se autoriza el ingreso de un vehículo, este debe retirarse de la lista de espera y registrarse en una lista doblemente enlazada.

Cada registro deberá almacenar:

- Placa y tipo de vehículo.
- Nombre del propietario.
- Fila y columna del espacio asignado.
- Identificador del encargado que autorizó el ingreso.

Operaciones obligatorias:

7. Incorporar un vehículo estacionado.
8. Recorrer los vehículos desde el primero hasta el último.
9. Recorrer los vehículos desde el último hasta el primero.
10. Buscar un vehículo mediante su placa.
11. Eliminar un vehículo cuando abandona el estacionamiento.
12. Consultar el total de vehículos estacionados.

La eliminación deberá funcionar independientemente de si el vehículo se encuentra al inicio, en medio o al final de la lista.

Importante: no es válido utilizar dos listas simples para simular una lista doble. Cada nodo deberá contar con sus referencias al nodo anterior y al siguiente.

3.3. Rotación de encargados — Lista enlazada circular

El estacionamiento cuenta con varios encargados responsables de autorizar el ingreso de vehículos. Para distribuir el trabajo equitativamente, se implementará un sistema de turnos rotativos.

Cada encargado deberá tener:

- Identificador único.
- Nombre.
- Cantidad de vehículos atendidos.

Operaciones obligatorias:

13. Registrar un encargado.
14. Eliminar un encargado.
15. Consultar quién atenderá la siguiente solicitud.

16. Asignar automáticamente un encargado cuando ingrese un vehículo.
17. Avanzar el turno al siguiente encargado.
18. Mostrar todos los encargados y sus cantidades de vehículos atendidos.

Ejemplo de rotación: Ana → Luis → María → Ana → Luis → María ...

Al ingresar un vehículo, se asigna el encargado del turno actual. Posteriormente, el sistema avanza al siguiente.

Si un encargado es eliminado, la estructura circular deberá conservar su funcionamiento. Deben contemplarse los casos en que la lista queda vacía o contiene un único nodo.

3.4. Administración de espacios — Matriz dinámica

El estacionamiento se representará mediante una matriz dinámica de enteros. Al iniciar el programa, se solicitará la cantidad de filas y columnas del estacionamiento.

La matriz deberá reservarse dinámicamente utilizando punteros y los operadores new y delete[].

Cada posición representa un espacio:

- 0: espacio libre.
- 1: espacio ocupado.

Operaciones obligatorias:

19. Crear la matriz según las dimensiones ingresadas.
20. Inicializar todos los espacios como libres.
21. Buscar el primer espacio disponible, recorriendo filas de izquierda a derecha.
22. Marcar un espacio como ocupado cuando ingresa un vehículo.
23. Liberar un espacio cuando sale un vehículo.
24. Mostrar visualmente el estado del estacionamiento.
25. Calcular el total de espacios disponibles y ocupados.
26. Calcular el porcentaje de ocupación.

Porcentaje de ocupación = (Espacios ocupados / Espacios totales) × 100

Por ejemplo, si hay 12 espacios y 9 están ocupados: (9 / 12) × 100 = 75 %.

Se deberá liberar toda la memoria reservada cuando finalice el programa.

3.5. Registro de ingreso y salida de vehículos

Este módulo deberá integrar las estructuras anteriores.

A. Registrar ingreso

Al procesar una solicitud pendiente, el sistema deberá:

27. Verificar que existan solicitudes.
28. Comprobar que haya al menos un espacio libre.
29. Comprobar que exista al menos un encargado.
30. Buscar el primer espacio disponible en la matriz.
31. Obtener el encargado correspondiente mediante la lista circular.
32. Registrar el vehículo en la lista doblemente enlazada.
33. Marcar el espacio de la matriz como ocupado.
34. Retirar la solicitud de la lista simple.

35. Actualizar la cantidad de vehículos atendidos por el encargado.
36. Avanzar al siguiente turno.

Si alguna condición impide el ingreso, la solicitud debe permanecer en espera y el resto del sistema debe conservar su estado.

B. Registrar salida

Cuando un vehículo abandona el estacionamiento:

37. Solicitar su placa.
38. Buscar el vehículo en la lista doble.
39. Solicitar la cantidad de horas de permanencia (número entero positivo).
40. Calcular el monto correspondiente.
41. Liberar la posición ocupada en la matriz.
42. Eliminar el vehículo de la lista doble.
43. Actualizar el total recaudado.
44. Mostrar un comprobante de salida.

Se utilizarán las siguientes tarifas de ejemplo:

Tipo de vehículo	Tarifa por hora
Automóvil	₡900
Motocicleta	₡500

Monto = Horas de permanencia × Tarifa por hora

Ejemplo: un automóvil permanece 3 horas. 3 × ₡900 = ₡2700.

No se requiere utilizar fechas ni calcular automáticamente el tiempo transcurrido.

3.6. Reportes del estacionamiento

El sistema deberá permitir consultar:

45. Total de vehículos en espera.
46. Total de vehículos estacionados.
47. Cantidad de espacios ocupados y libres.
48. Porcentaje de ocupación.
49. Cantidad de vehículos atendidos por cada encargado.
50. Total de dinero recaudado.
51. Cantidad de automóviles y motocicletas actualmente estacionados.

Los reportes deberán obtener su información de las estructuras implementadas, no de arreglos adicionales que dupliquen los registros.

4. Implementación de iteradores

Se deberán implementar iteradores propios para recorrer las tres listas enlazadas. No se permite sustituir estos iteradores por recorridos realizados exclusivamente mediante índices o por acceso directo a los punteros de los nodos desde main().

4.1. Iterador de la lista simple

Deberá permitir las siguientes operaciones:

Operación	Función
operator*	Obtener el dato almacenado en el nodo actual.
operator++	Avanzar al siguiente nodo.
operator!=	Comparar dos iteradores.
begin()	Obtener el iterador inicial.
end()	Obtener el iterador que representa el final.

Su uso deberá permitir un recorrido como:

```
for (auto it = lista.begin();
    it != lista.end(); ++it) {
   cout << (*it).getPlaca() << endl;
}
```

4.2. Iterador de la lista doble

Además de las operaciones anteriores, deberá incorporar operator--, para retroceder al nodo anterior.

Se deberán demostrar recorridos hacia adelante y hacia atrás, utilizando métodos apropiados para obtener las posiciones de inicio y final de cada recorrido.

4.3. Iterador de la lista circular

Se deberá implementar un mecanismo de iteración que permita recorrer todos los encargados exactamente una vez.

Importante: debido a que la lista es circular, el iterador deberá contar con una condición de finalización que evite ciclos infinitos. Puede utilizarse un contador de elementos pendientes o un indicador que permita terminar después de completar una vuelta.

4.4. Restricciones de los iteradores

- operator* debe retornar una referencia al dato, no obligar al usuario a manipular el nodo directamente.
- Los iteradores deben utilizarse al menos en las funciones de listado y en una operación de cálculo o búsqueda.
- No se debe desreferenciar un iterador que haya llegado al final.
- Los recorridos deberán funcionar también con listas vacías.
- No se deberán modificar las estructuras mientras se recorren con un iterador, salvo cuando una operación esté diseñada específicamente para hacerlo de manera segura.

5. Manejo de excepciones en C++

5.1. ¿Qué es una excepción?

Una excepción es un mecanismo que permite indicar que ocurrió una situación que impide completar una operación normalmente.

Por ejemplo, al intentar ingresar un vehículo, pueden presentarse las siguientes situaciones:

- El estacionamiento está lleno.
- No existen encargados disponibles.
- La placa ya está registrada.

- No hay solicitudes pendientes.
- Se solicita una posición inexistente en la matriz.

Sin excepciones, estas situaciones podrían provocar errores o producir resultados incorrectos si no se controlan adecuadamente.

En C++, las excepciones permiten separar el código que realiza una operación del código encargado de gestionar los errores.

5.2. ¿Qué es throw?

La instrucción throw permite lanzar una excepción cuando se detecta una situación inválida. Por ejemplo:

```
void registrarHoras(int horas) {
   if (horas <= 0) {
      throw invalid_argument(
         "Las horas deben ser positivas"
      );
   }
}
```

Cuando se ejecuta throw, la función interrumpe su ejecución normal y la excepción se propaga hasta encontrar un bloque que pueda capturarla.

5.3. ¿Qué son try y catch?

En C++, try y catch permiten ejecutar operaciones que podrían producir excepciones y responder adecuadamente cuando ocurren.

try: contiene las instrucciones que podrían lanzar una excepción.

catch: captura una excepción del tipo correspondiente y define qué hacer cuando se produce.

Ejemplo:

```
#include <iostream>
#include <stdexcept>
using namespace std;

int main() {
   int horas;
   cout << "Ingrese las horas: ";
   cin >> horas;

   try {
      if (horas <= 0) {
         throw invalid_argument(
            "Las horas deben ser mayores que cero"
         );
      }
      cout << "Horas registradas: " << horas;
   }
   catch (const invalid_argument& e) {
      cout << "Error: " << e.what() << endl;
   }

   cout << "\nFin de la operacion.";
   return 0;
}
```

Si el usuario ingresa -3, el programa mostrará:

```
Ingrese las horas: -3
Error: Las horas deben ser mayores que cero

Fin de la operacion.
```

Observe que el programa no ejecuta las instrucciones posteriores al throw dentro del bloque try. En cambio, se dirige al catch correspondiente. La expresión e.what() permite obtener el mensaje asociado a la excepción.

Nota: este ejemplo supone que el usuario introduce un número entero. Una entrada no numérica mediante cin requiere validación adicional; no genera automáticamente una excepción invalid_argument.

5.4. Tipos de excepciones

Para este proyecto se utilizará la biblioteca <stdexcept>.

Excepción	Ejemplo de aplicación
invalid_argument	Horas negativas, dimensiones inválidas o placa vacía.
out_of_range	Consulta de una posición inexistente.
overflow_error	Intento de ingresar un vehículo cuando no quedan espacios.
underflow_error	Intento de procesar una lista de espera vacía.
logic_error	Intento de asignar un vehículo cuando no existen encargados o cuando la placa ya está registrada.

5.5. Requisitos obligatorios

52. Lanzar excepciones en al menos cinco situaciones diferentes del sistema.

53. Capturar las excepciones mediante bloques try/catch.
54. Mostrar mensajes comprensibles para el usuario.
55. Permitir que el menú principal continúe funcionando después de capturar un error recuperable.
56. Evitar que una operación fallida deje información inconsistente.

No se permite reemplazar todas las excepciones con mensajes de cout o finalizar el programa utilizando exit() cada vez que ocurre un error.

La validación debe efectuarse antes de modificar información importante, especialmente en las operaciones que afectan varias estructuras.

6. Menú principal del sistema

El programa deberá contar con un menú interactivo que permita acceder a las distintas funcionalidades.

```
========================================
  SISTEMA DE ESTACIONAMIENTO
========================================
1. Gestionar solicitudes de ingreso
2. Procesar siguiente solicitud
3. Consultar vehiculos estacionados
4. Registrar salida de vehiculo
5. Gestionar encargados
6. Mostrar matriz del estacionamiento
7. Generar reportes
0. Salir

Seleccione una opcion:
```

Las opciones 1, 3 y 5 deberán contar con submenús para ejecutar todas las operaciones solicitadas.

El programa deberá continuar mostrando el menú hasta que se seleccione la opción de salida.

Se deberán utilizar ciclos, condiciones y funciones para controlar la navegación y validar las operaciones.

7. Ejemplo de ejecución

La siguiente ejecución integra las operaciones principales del proyecto. Se muestra una secuencia continua de uso; para facilitar la lectura, no se repite el menú completo después de cada operación. Los mensajes y nombres de opciones pueden variar, siempre que el comportamiento solicitado se conserve.

7.1. Creación del estacionamiento y registro de encargados

```
CONFIGURACION INICIAL
Ingrese cantidad de filas: 2
Ingrese cantidad de columnas: 2
Estacionamiento creado correctamente.
Espacios disponibles: 4

GESTION DE ENCARGADOS
ID: E01   Nombre: Ana
ID: E02   Nombre: Luis
ID: E03   Nombre: Maria
Encargados registrados: 3
Orden de rotacion: Ana -> Luis -> Maria -> Ana
Proximo encargado: Ana
```

En este ejemplo, el estacionamiento tiene cuatro espacios; por ello, puede alcanzarse su capacidad máxima con pocos ingresos.

7.2. Solicitudes, búsqueda, duplicados y cancelación intermedia

```
REGISTRAR SOLICITUD
S001  ABC123  Carlos   Automovil
S002  MOTO45  Daniela  Motocicleta
S003  XYZ789  Pedro    Automovil
S004  JKL555  Sofia    Automovil

CONSULTAR SOLICITUD POR PLACA
Placa: MOTO45
Resultado: S002 - Daniela - Motocicleta

REGISTRAR SOLICITUD
Placa: ABC123
Error (logic_error): La placa ya esta registrada.

CANCELAR SOLICITUD
Placa: XYZ789
Solicitud S003 cancelada correctamente.

LISTA DE ESPERA - RECORRIDO CON ITERADOR
S001 ABC123  Carlos
S002 MOTO45  Daniela
S004 JKL555  Sofia
Total de solicitudes pendientes: 3
```

La cancelación de XYZ789 elimina un nodo intermedio sin alterar el orden de las solicitudes restantes. El intento de registrar una placa duplicada no modifica la lista.

7.3. Primeros ingresos y recorrido bidireccional

```
PROCESAR SIGUIENTE SOLICITUD
Solicitud: S001 - ABC123
Espacio asignado: Fila 1, Columna 1
Encargado: Ana
Ingreso registrado correctamente.
Siguiente encargado: Luis

PROCESAR SIGUIENTE SOLICITUD
Solicitud: S002 - MOTO45
Espacio asignado: Fila 1, Columna 2
Encargado: Luis
Ingreso registrado correctamente.
Siguiente encargado: Maria

VEHICULOS ESTACIONADOS - ORDEN DIRECTO
ABC123  (1,1)  Ana
MOTO45  (1,2)  Luis

VEHICULOS ESTACIONADOS - ORDEN INVERSO
MOTO45  (1,2)  Luis
ABC123  (1,1)  Ana
```

Ambos recorridos deberán utilizar los iteradores desarrollados para la lista doblemente enlazada.

7.4. Ocupación completa de la matriz

```
REGISTRAR SOLICITUD
S005  NEW777  Esteban  Automovil

PROCESAR SIGUIENTE SOLICITUD
Solicitud: S004 - JKL555
Espacio asignado: Fila 2, Columna 1
Encargado: Maria
Siguiente encargado: Ana

PROCESAR SIGUIENTE SOLICITUD
Solicitud: S005 - NEW777
Espacio asignado: Fila 2, Columna 2
Encargado: Ana
Siguiente encargado: Luis

ESTADO DEL ESTACIONAMIENTO
     C1   C2
F1     O    O
F2     O    O
O = Ocupado  |  L = Libre
Espacios ocupados: 4
Espacios libres: 0
Porcentaje de ocupacion: 100.00%
```

7.5. Estacionamiento lleno y recuperación sin pérdida de datos

```
REGISTRAR SOLICITUD
S006  ESP888  Andrea  Motocicleta
Solicitud registrada correctamente.

PROCESAR SIGUIENTE SOLICITUD
Error (overflow_error): No hay espacios disponibles.

LISTA DE ESPERA
S006 ESP888 Andrea Motocicleta
Pendientes: 1

CONSULTAR SIGUIENTE ENCARGADO
Luis
```

La solicitud continúa en espera y el turno de Luis no cambia, porque el ingreso no se completó.

7.6. Salida con error, cálculo de tarifa y liberación

```
REGISTRAR SALIDA
Placa: ABC123
Propietario: Carlos
Tipo: Automovil
Horas de permanencia: -3
Error (invalid_argument): Las horas deben ser positivas.

CONSULTAR PLACA
ABC123 sigue estacionado en (1,1).

REGISTRAR SALIDA
Placa: ABC123
Horas de permanencia: 3
Tarifa por hora: 900 colones
Monto a pagar: 2700 colones
Espacio (1,1) liberado.
Vehiculo retirado del registro.

ESTADO DEL ESTACIONAMIENTO
     C1   C2
F1     L    O
F2     O    O
Espacios libres: 1
Porcentaje de ocupacion: 75.00%
```

El intento inválido no afecta el vehículo ni la matriz. La salida correcta libera su espacio, actualiza la lista doble y registra ₡2700 en el total recaudado.

7.7. Ingreso de la solicitud pendiente y nueva rotación

```
PROCESAR SIGUIENTE SOLICITUD
Solicitud: S006 - ESP888
Espacio asignado: Fila 1, Columna 1
Encargado: Luis
Ingreso registrado correctamente.
Siguiente encargado: Maria

VEHICULOS ESTACIONADOS - ORDEN DIRECTO
MOTO45  (1,2)  Luis
JKL555  (2,1)  Maria
NEW777  (2,2)  Ana
ESP888  (1,1)  Luis

VEHICULOS ESTACIONADOS - ORDEN INVERSO
ESP888  (1,1)  Luis
NEW777  (2,2)  Ana
JKL555  (2,1)  Maria
MOTO45  (1,2)  Luis
```

Se reutiliza el primer espacio disponible, y la nueva entrada se coloca al final de la lista doble. El total de vehículos estacionados vuelve a ser cuatro.

7.8. Reportes integrados y operaciones adicionales

```
REPORTE GENERAL
Solicitudes pendientes: 0
Vehiculos estacionados: 4
Automoviles estacionados: 2
Motocicletas estacionadas: 2
Espacios ocupados: 4
Espacios libres: 0
Porcentaje de ocupacion: 100.00%
Total recaudado: 2700 colones

VEHICULOS ATENDIDOS POR ENCARGADO
E01 Ana: 2
E02 Luis: 2
E03 Maria: 1

REGISTRAR SOLICITUD
Placa: MOTO45
Error (logic_error): La placa ya esta registrada.

REGISTRAR SALIDA
Placa: NOEXISTE
Error (out_of_range): Vehiculo no encontrado.
```

El conteo de atenciones es acumulado: incluye también el ingreso de ABC123, aunque ese vehículo ya salió. La cantidad de vehículos estacionados se calcula únicamente a partir de los registros activos.

7.9. Eliminación circular, cancelación y lista vacía

```
GESTION DE ENCARGADOS
Eliminar encargado: E02
Encargado Luis eliminado.
Rotacion actual: Ana -> Maria -> Ana
Proximo encargado: Maria

REGISTRAR SOLICITUD
S007 TEMP100 Roberto Automovil
Solicitud registrada correctamente.

CANCELAR SOLICITUD
Placa: TEMP100
Solicitud S007 cancelada correctamente.
Pendientes: 0

PROCESAR SIGUIENTE SOLICITUD
Error (underflow_error): No hay solicitudes pendientes.

CONSULTAR LISTA DE ESPERA
No existen solicitudes pendientes.

FINALIZAR PROGRAMA
Memoria liberada. Programa finalizado.
```

La eliminación de un encargado no debe invalidar los registros históricos de ingresos: los vehículos conservan el identificador del encargado que los atendió. El recorrido de la lista circular continúa sin ciclos infinitos.

Además de los casos anteriores, el programa deberá controlar dimensiones inválidas de la matriz, falta de encargados, posiciones fuera del rango y listas vacías, de acuerdo con los requerimientos del proyecto.

8. Requisitos técnicos

8.1. Programación orientada a objetos

El proyecto deberá organizarse mediante clases con responsabilidades claramente definidas.

Como mínimo, deberán existir clases que representen:

- Solicitud de ingreso.
- Vehículo estacionado.
- Encargado.
- Lista enlazada simple.
- Lista doblemente enlazada.
- Lista enlazada circular.
- Matriz dinámica.
- Sistema de administración.

Cada lista deberá implementar sus propios nodos y las operaciones necesarias para su funcionamiento.

Se podrán utilizar clases adicionales cuando resulten necesarias.

8.2. Memoria dinámica

Los nodos de las listas y los elementos de la matriz deberán administrarse dinámicamente.

Se deberán utilizar correctamente:

- Punteros.
- new y delete.
- new[] y delete[].
- Constructores y destructores.

Cada estructura deberá liberar la memoria que administra.

Para evitar copias superficiales de estructuras que poseen memoria dinámica, las clases deberán implementar correctamente la copia profunda o prohibir la copia cuando no sea necesaria.

8.3. Restricciones

57. No se permite utilizar std::list, std::forward_list, std::queue, std::deque ni std::vector para sustituir las estructuras solicitadas.
58. La matriz no puede implementarse mediante arreglos de dimensiones fijas.
59. Los nodos y las listas deberán implementarse de forma propia.
60. No se permite resolver todos los recorridos mediante punteros utilizados directamente desde main().
61. No se permite almacenar los vehículos estacionados en una estructura adicional que sustituya la lista doble.
62. El programa deberá estar dividido en funciones y clases; no se permite implementar toda la lógica dentro de main().
63. No se requiere interfaz gráfica ni conexión con bases de datos.
64. Los datos se almacenarán en memoria durante la ejecución; no se requiere persistencia en archivos.

9. Pruebas del sistema

Cada estructura deberá probarse por separado antes de verificar su integración con el resto del sistema.

Como mínimo, deberán contemplarse los siguientes casos:

N.º	Caso de prueba	Resultado esperado
1	Registrar una solicitud válida	Se incorpora al final de la lista simple.
2	Cancelar la primera solicitud	Se actualiza correctamente el inicio.
3	Cancelar una solicitud intermedia	Se conserva la continuidad de la lista.
4	Eliminar un vehículo de la lista doble	Se actualizan los enlaces anterior y siguiente.
5	Recorrer una lista doble en ambos sentidos	Se obtienen los registros en el orden correcto.
6	Asignar cuatro vehículos con tres encargados	La rotación regresa al primer encargado.
7	Eliminar un encargado de una lista circular	La rotación continúa correctamente.
8	Ocupar todos los espacios de la matriz	El porcentaje alcanza el 100 %.
9	Intentar ingresar otro vehículo con el estacionamiento lleno	Se captura una excepción y la solicitud permanece pendiente.

N.º	Caso de prueba	Resultado esperado
10	Registrar horas negativas	Se rechaza la operación mediante una excepción.
11	Registrar una placa duplicada	Se rechaza el registro.
12	Retirar un vehículo	Se libera su espacio y se actualiza la lista doble.
13	Procesar una lista de espera vacía	Se captura la excepción correspondiente.
14	Consultar una lista vacía mediante iteradores	El recorrido termina sin acceder a memoria inválida.

El proyecto deberá incluir al menos 10 pruebas automatizadas utilizando assert o funciones de prueba equivalentes. También se entregará evidencia de pruebas de integración realizadas desde el menú.

10. Entregables

Primer avance (final de la primera semana):

- Código fuente de las dos primeras listas.
- Implementación inicial de los iteradores.
- Diagrama de clases o representación equivalente.
- Programa de prueba de las operaciones desarrolladas.

Entrega final (final de la segunda semana):

65. Proyecto completo de C++, con sus archivos .h y .cpp, cuando corresponda.
66. Programa funcional con menú interactivo.
67. Código de las pruebas implementadas.
68. Documento breve que describa la organización de las clases, las decisiones de diseño y los resultados de las pruebas.
69. Instrucciones para compilar y ejecutar.
70. Demostración del funcionamiento del programa.

En la modalidad en parejas, ambos integrantes deberán comprender el funcionamiento de todas las estructuras, independientemente de cómo hayan distribuido su implementación.

11. Evaluación

Criterio	Porcentaje
Lista enlazada simple y operaciones	12 %
Lista doblemente enlazada y operaciones	15 %
Lista circular y rotación de encargados	13 %
Matriz dinámica y gestión de memoria	15 %
Implementación y utilización de iteradores	15 %
Manejo de excepciones	10 %
Integración de módulos, ciclos, condiciones y cálculos	10 %
Pruebas y validación de resultados	5 %
Organización del código y documentación	5 %
TOTAL	100 %

12. Consideraciones finales

El objetivo del proyecto no consiste únicamente en implementar estructuras de datos de manera aislada, sino en comprender cómo pueden colaborar para resolver un problema.

Se valorará especialmente que:

- Las estructuras mantengan su integridad después de las inserciones y eliminaciones.
- Los iteradores permitan recorrer las listas sin exponer innecesariamente sus nodos.
- El sistema controle adecuadamente sus errores.
- No existan pérdidas de memoria por falta de liberación.
- Los módulos estén correctamente integrados.
- Los cálculos y reportes sean consistentes con la información almacenada.

El proyecto se considerará completo cuando todas las funcionalidades obligatorias puedan ejecutarse desde el programa y sus resultados puedan verificarse mediante pruebas.